

SecureQNN: Shielding the Intellectual Property of QNNs in TinyML Systems

Miguel Costa, Tiago Gomes, Sandro Pinto

Abstract—Building accurate Machine Learning (ML) models requires substantial expertise and large-scale datasets typically only available in big data companies. These companies have been selling their models as Machine Learning as a Service. However, concerns about data privacy and the appliance of ML in mission-critical scenarios are forcing ML computation to move from the cloud to the deep edge, near sensor data. If edge ML increases user data privacy and makes decision latency predictable, deploying proprietary ML models on untrusted edge devices may harm the intellectual property of Service Providers. A natural response to this issue comes from Trusted Execution Environments (TEEs), which provide hardware-based security. However, adapting ML computation to the constraints of TEEs remains an open challenge. In this article, we propose SecureQNN, a framework that leverages state-of-the-art TEE technology (TrustZone-M) available in Arm Cortex-M microcontrollers to increase the protection of Quantized Neural Networks (QNNs) against unauthorized replication. SecureQNN evaluates which layers lessen the effort of an attacker when building a surrogate QNN with the same accuracy. The layers making the attacker spend less training epochs than the owner training from scratch are stored and executed in the secure-world of the TEE. Experiments demonstrate that SecureQNN undermines the cost-benefit of unauthorized QNN replication by isolating 51%–65% of the model size while incurring a worst-case decision latency overhead of only 0.06%. Although SecureQNN has broader applicability, the negligible impact on decision latency and its deterministic behavior highlight the suitability of SecureQNN for mission-critical and real-time applications.

Index Terms—TinyML, Quantized Neural Networks, ML Privacy, TEE, TrustZone, Arm Cortex-M.

I. INTRODUCTION

Developing accurate and efficient Machine Learning (ML) models requires substantial expertise and large-scale datasets, typically unavailable to ML clients. Consequently, this task is predominantly undertaken by major technology companies such as Amazon and Google, which sell pre-trained models via Machine Learning as a Service (MLaaS) [1, 2]. While MLaaS enables clients to utilize third-party models for inference on private data, it shows several limitations. Traditional MLaaS follows a centralized computing paradigm, requiring clients to transmit private data to cloud servers, necessitating trust in the server and the network infrastructure. Furthermore, frequent data transfers can congest networks and cloud computation units, leading to problematic delays for real-time applications, such as autonomous systems, medical diagnostics, and forest fire detection [3–5].

The authors are with the ALGORITMI Research Centre of University of Minho - Campus de Azurém, in 4800-058, Guimarães, Portugal (e-mail: miguel.costa@dei.uminho.pt).

Manuscript received January 8, 2025; revised X X, 2025.

These growing concerns over data privacy and mission-critical ML, aligned with advances in memory and processing resources in low-power Internet of Things (IoT) devices led to the ongoing trend to shift ML computations to the edge of the network [1, 2, 4–10]. Although deploying ML models on user devices enhances privacy by eliminating the need to transfer user data to third parties, deploying such models on untrusted devices may damage the intellectual property of the Service Provider (SP). Unauthorized access to these models can expose proprietary algorithms by reverse engineering [11] or leak private training data [11–13]. Therefore, researchers from academia and industry have been investigating mechanisms to transfer a proprietary ML model to the edge while providing privacy guarantees to the model owner (SP).

The most prominent solutions to date rely on Trusted Execution Environments (TEEs) available on Commercial-Off-The-Shelf (COTS) application processors (APUs). TEEs offer built-in hardware primitives to enforce memory isolation between applications running in a Rich Execution Environment (REE) and critical applications [14]. However, applications within a TEE must be as small as possible to minimize the Trusted Computing Base (TCB), requiring the redesign of ML systems.

In recent years, several TEE-based approaches have been proposed to increase the intellectual property protection of ML models in untrusted devices. Most works [1, 7–9] encrypt entire Artificial Neural Networks (ANNs) and store them in unprotected memory. To minimize the TCB, the ANN is loaded layer-by-layer from non-secure to secure memory. Within the TEE, the layer is decrypted and the activations are calculated. Despite their novelty, these works incur increased latency due to frequent cryptography operations and context switches between normal and secure worlds. More efficient works (i) perform model loading and prediction in parallel [15]; (ii) extend TEEs [10, 16], (iii) execute only strategic layers in the TEE [6, 17–19]; (iii) obfuscate strategic ANN weights [2]; or (iv) refactor ANNs in a bident structure [20].

Despite these advancements, existing solutions primarily target x86 and Arm Cortex-A APUs, neglecting Arm Cortex-M microcontrollers (MCUs), projected to reach one trillion units by 2035 [21] and already powering most IoT applications. Recent support for Single Instruction Multiple Data (SIMD) for the execution of Quantized Neural Networks (QNNs) [21] and the availability of developer-friendly ML frameworks (CMSIS-NN) [21, 22] increased Cortex-M's relevance in ML research [23–25]. The introduction of TrustZone-M in Armv8-M further raises the question of whether TEEs can effectively protect or increase the protection of the intellectual property

of QNNs on these resource-constrained devices.

Arm Cortex-M MCUs only hold a few megabytes of memory and deliver substantially lower throughput than Cortex-A and x86 processors. Therefore, porting works that perform full ANN decryption within the TEE [1, 7–9, 15] is inefficient due to the overhead of frequent cryptography operations. Cortex-M MCUs do not support Graphics Processing Unit (GPU) acceleration to compensate for this overhead. Refactoring an ANN in a bident structure [20] is also problematic, as Arm Cortex-M devices only feature a single core, not allowing the parallel execution of two different ANNs. Extending TEEs with custom Memory Management Units (MMUs) [10, 16] is infeasible too, as Cortex-M lacks MMU support. Weight obfuscation [2] is more efficient but still imposes considerable overhead to denoise the activations of each ANN layer.

A more viable approach is executing only strategic layers within the secure-world while running the remaining layers in the normal-world. However, existing solutions in this category are also unsuitable for Cortex-M: Slalom [17] and ShadowNet [18] require GPUs, DarkneTZ [6] protects training data rather than the ANN itself, and SecureFrameNet [19] relies on Floating Point Operations (FLOPs) to balance the TEE workload, whereas Cortex-M typically runs ANNs in integer precision (QNNs). These limitations lead to a pivotal question: *can TEEs be leveraged to protect the privacy of QNNs on resource-constrained Arm Cortex-M MCUs?*

To answer this research question, we propose SecureQNN: a framework to determine the most critical layers of a QNN and protect their privacy using Arm TrustZone-M. Our framework traverses a given QNN layer-by-layer and determines which set of consecutive layers is privacy-critical and must be delegated to the secure-world. Our framework simulates the most challenging threat model, where the attacker has access to everything in the normal-world and can use the available information to recover the full model. The attacker is also aware of the architecture of the private layers. In this scenario, the attacker can freeze the public layers and train a substitute model till it gets an accuracy equal to or higher than the accuracy of the original model. SecureQNN is designed to undermine the cost-benefit of unauthorized QNN replication. We consider a QNN to be sufficiently secure when the effort required by the attacker to train a substitute model is, at least, equal to the effort needed for the SP to train the original model from scratch. We measure the effort in training epochs. Additionally, SecureQNN leverages Trusted Firmware-M (TF-M) - a reference implementation of secure firmware for Cortex-M MCUs - to protect the integrity of QNNs and prevent the success of highly advanced attacks probing the flash memory.

We evaluated SecureQNN using QNNs and datasets from the MLPerf Tiny benchmark [25], which reflects real-world TinyML applications identified by over 30 academic and industrial organizations. Our evaluation focuses on two key metrics: (i) TEE memory footprint and (ii) inference latency. Results show that the cost-benefit of unauthorized QNN replication can be undermined by delegating only a small fraction of the QNN layers to the secure-world. Regarding latency, measurements conducted on an STM Nucleo-L552ZE-

Q board, featuring an Arm Cortex-M33 at 110 MHz, indicate that SecureQNN introduces negligible overhead, maintaining deterministic and predictable inference performance suitable for real-time edge applications. All artifacts are open-sourced to enable independent validation of results and encourage further exploration of techniques to protect the intellectual property of QNNs deployed in edge devices¹.

In short, this work makes the following contributions: (i) the first public and open-source framework aimed at increasing the protection of the intellectual property of QNNs deployed in Arm Cortex-M MCUs; (ii) a novel strategy to determine the set of privacy-critical layers in an ANN/QNN; (iii) the concept of default privacy setting, whereby the intellectual property of an ANN/QNN is considered sufficiently secure when attackers have to pay the same effort as the model owner training from scratch to get a surrogate ANN/QNN with equal or higher accuracy, thus reducing the cost-benefit and likelihood of successful model replication by third parties.

Our framework (SecureQNN) was first introduced as a conference perspective paper [26]. This earlier publication [26] focused on (i) analyzing existing literature, (ii) highlighting the importance of the topic, (iii) defining the threat model, and (iii) providing a high-level overview of SecureQNN.

In this new publication, we expand on the initial work significantly. In addition to the extension and refinement of previous topics, the current paper:

- 1) Describes the strategy to simulate the creation of a surrogate dataset by a powerful attacker (Section IV-A) and presents a comprehensive qualitative and quantitative evaluation of the surrogate datasets (Section V-A);
- 2) Extends the model partitioning strategy with new functionality to achieve a finer trade-off between TEE memory footprint and privacy (Section IV-B);
- 3) Restructures the Trusted Execution module to address initial design flaws and introduces protection against flash-probing attacks (Section IV-C).
- 4) Details the TEE memory footprint across different privacy levels and development boards (Section V-B);
- 5) Analyzes the decision latency (Section V-C), including the latency of the context switches and the overhead resulting from the protection against flash-probing attacks;
- 6) Discusses the current limitations of the framework and potential directions for future work (Section VI).

II. METHODOLOGY

The methodology guiding the design and evaluation of SecureQNN is grounded in two core aspects: the framework scope and the threat model. The scope defines the target hardware computing platform and software libraries leveraged by our framework, while the threat model formalizes the adversarial assumptions and security goals.

A. Scope

This work targets ultra-low-power IoT devices powered by Arm Cortex-M MCUs following the latest Armv8-M

¹<https://github.com/MiguelCosta94/SecureQNN.git>

and Armv8.1-M architectures and supported by TF-M. These devices typically feature a single core, alongside flash and Static Random Access Memory (SRAM). Both flash and SRAM sizes vary between hundreds of kilobytes and a few megabytes. These MCUs execute instructions directly from physical memory and do not feature an MMU for virtual memory support, making them incapable of running Unix-like operating systems. While these MCUs offer various peripherals, they do not support GPU acceleration. Advanced Armv8-M and Armv8.1-M MCUs feature TrustZone-M technology, allowing developers to establish two execution environments: secure and normal.

CMSIS-NN and TensorFlow Lite Micro, built on top of CMSIS-NN, are state-of-the-art libraries for running QNNs on Arm Cortex-M MCUs. CMSIS-NN is accelerated by SIMD for matrix and convolution operations with 8-bit and 16-bit operands. In our framework, we use CMSIS-NN instead of TensorFlow Lite Micro to reduce the memory footprint of the secure-world. CMSIS-NN supports int-8 and int-16 QNNs with fixed bit-width and following a (i) uniform, (ii) static, and (iii) symmetric quantization. For more details on the CMSIS-NN library, we refer readers to work [21].

B. Threat Model

Figure 1 summarizes the threat model of SecureQNN. It encompasses two agents: (i) the SP or model owner and (ii) an honest but curious client.

a) Service Provider (SP): The SP uses its capacity to store and process data to train a proprietary ML model. The trained model constitutes the intellectual property of the SP and the SP is interested in maintaining it secret. In addition, the SP is also concerned with the privacy of the training data. The training data could be generated by the SP itself, or provided by a third party under a given confidentiality agreement. Therefore, leaking training data would not only undermine users' trust in the SP but could also put the SP in legal action raised by the data owner. Considering that recent studies have shown that ML models are vulnerable to the so-called Membership Inference Attacks (MIAs) - attacks specifically designed to understand if a given data point belongs to the training dataset - the SP is also interested in reducing the success rate of this type of attack to keep the training data private. Storing and running the entire QNN in the secure-world is not feasible. Therefore, the SP deploys the ANN in

clients' devices as a gray-box, where part of it is executed within the secure-world and another in the normal-world. The SP attests to the privacy of the model when the effort (number of epochs) required by the attacker to construct a surrogate model with an accuracy equal to or higher than the accuracy of the original model is, at least, equal to the effort of the SP to train the original model from scratch.

b) Client: The client gets the QNN owned by the SP on its local device powered by an Arm Cortex-M MCU as long as it supports the Arm TrustZone-M and TF-M technologies. The client is an honest but curious adversary: they follow the protocol specified by the SP but are interested in learning about the QNN. Regarding capability and knowledge, we assume a worst-case scenario. The adversary can manipulate everything that runs in the normal-world, including the parameters of public QNN layers and the operating system. Also, the client is aware of the architecture of private layers and the training hyper-parameters. Except for the last QNN private layer, which is typically a fully-connected layer with softmax activation, in a real-world scenario, it is hard to access the architecture of private layers and training hyper-parameters. Nevertheless, TEEs are not bulletproof [27], and, therefore, a worst-case scenario increases the level of privacy protection.

An advanced adversary can also probe the flash memory to retrieve the content stored in secure memory. However, this type of attack is extremely difficult to perform in modern Systems on Chip (SoCs) and requires expensive equipment. Other physical attacks are out of the scope of our work. The client has access to the original testing dataset but not the training data. Considering that data is the most valuable asset of any ML model, if the attacker already has access to the training data, there is no interest in stealing the model.

III. BACKGROUND AND RELATED WORK

This section provides an overview of the concepts and key technologies that support the development of SecureQNN. We begin by discussing Arm TrustZone and its adaptation for MCUs (TrustZone-M). We then explore TinyML, a growing field focused on deploying ML on ultra-low-power devices, and delve into Generative Adversarial Networks (GANs), a key ML technology to simulate the creation of a surrogate dataset by a potential attacker. We end with a comprehensive literature review on solutions addressing the topic of intellectual property/privacy of ML models on edge devices and how our work advances the state-of-the-art.

A. Arm TrustZone(-M)

Arm TrustZone was introduced in 2004 as a hardware security extension of Arm APUs (Arm Cortex-A) [28]. TrustZone follows a system-wide approach to security, providing hardware-enforced protection mechanisms at the Central Processing Unit (CPU) and SoC levels. Initially deployed in the Armv6K architecture, this technology imposed important architectural changes, introducing two protection domains: (i) the secure-world and (ii) the normal-world [14, 28]. The secure-world hosts privacy- and integrity-critical software

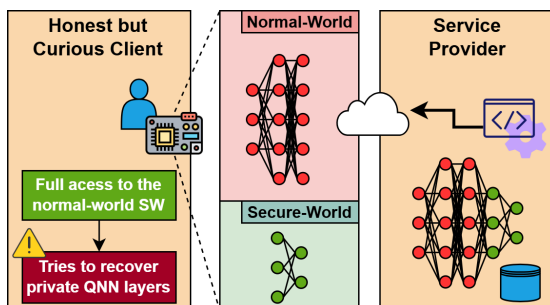


Fig. 1. SecureQNN threat model: the client of the SP is honest but curious, trying to recover the private QNN layers.

components, while the normal-world runs less sensitive applications. At a given moment, the processor operates exclusively in one world. In Arm Cortex-A processors, the CPU status is defined by the Non-Secure (NS) bit of the Secure Configuration Register (SCR) [14, 28]. The context switch is handled by a privileged software mechanism called Secure Monitor, which saves the state (registers, stack pointers, etc.) of the current world before switching to the other [14, 28].

In 2016, TrustZone was adapted to the new generation of Arm MCUs, namely those based on Armv8-M and Armv8.1-M architectures [14]. The newer technology, Arm TrustZone-M, follows the same dual-world architecture, with non-secure software blocked from directly accessing secure components. However, at the low level, there are significant changes as Cortex-M is optimized for faster context switch and low-power applications [14, 28].

In contrast to Arm Cortex-A APU, in Armv8-M and Armv8.1-M the security state does not depend on a specific security bit. The division between the normal and secure worlds is memory-based: when the MCU operates from secure memory, it is in a secure state, and when operating from non-secure memory, it is in a normal state [14, 28]. To determine the secure state of the MCU, TrustZone-M relies on two distinct units: (i) the Security Attribution Unit (SAU) and (ii) the Implementation-Defined Attribution Unit (IDAU) [14, 28, 29]. The former is programmable, enabling dynamic memory partitioning; the latter sets static partitions determined by the chip manufacturer. TrustZone-M determines the security status of an address by performing a logical OR on the outputs from the SAU and IDAU. In contrast to Cortex-A, Armv8-M and Armv8.1-M exclude the need for a secure monitor software to perform the context switch [14, 28]. The bridge between both worlds is handled by a set of mechanisms implemented into core logic. For more information about Arm TrustZone-M, we refer readers to works [14, 28, 29].

B. TinyML: The new frontier in AI?

a) What is TinyML?: TinyML is a branch of ML that focuses on developing hardware and software solutions that enable the deployment of lightweight ML computation in devices consuming 1 milliWatt or less [25], such as ultra-low-power MCUs. TinyML emerged as a response to the increasing concerns about data privacy, but also to the cloud and network limitations disrupted by the boom of IoT devices.

b) TinyML in the Market: Arm predicts that by 2035, MCUs could surpass 1 trillion across multiple market segments [21]. In the same line, a market study [30] projects that the MCU market size is expected to reach \$992.80 million by 2030. Regarding the TinyML market, ABI Research forecasts that it will expand from 15.2 million shipments in 2020 to 2.5 billion by 2030 [31] and TinyML Software-as-a-Service could potentially become a billion-dollar market by 2030 [32].

c) TinyML in Academia: In academia, multiple software and hardware solutions have been proposed to enable and optimize TinyML. From the myriad of proposed solutions, TinyML has played a pivotal role in the increasing relevance of RISC-V Instruction Set Architecture (ISA). Flamand et al.

[33] proposed GAP8 - an SoC tailored to optimize the execution of QNNs - competing with the already established Arm Cortex-M MCUs. This SoC became so relevant that GreenWaves Technologies started mass production and even proposed a newer iteration (GAP9) [34]. To further support the adoption of GAP8, Garofalo et al. [35] developed PULP-NN, a software library optimized for the execution of 8-bit QNNs. AskariHemmat et al. [36] went a step further and proposed a RISC-V vector processor tailored to perform inference using sub-byte QNNs. Hoyer et al. [37] brought these innovations to the real-world, proposing a RISC-V device running QNNs to monitor and detect heart arrhythmia.

d) TinyML Use Cases: In terms of use cases, TinyML systems were initially designed for (i) word spotting, (ii) object recognition, (iii) object counting, and (iv) audio/voice detection [25, 32]. Recently, TinyML systems have been extended to (i) forest fire detection, (ii) health monitoring, (iii) anomaly detection, and (iv) smart pharming, among others [32]. For more information on the relevance and use cases of TinyML, we refer the readers to surveys [38–40].

C. Generative Adversarial Networks (GANs)

GANs are a class of deep learning frameworks commonly used to generate new data that closely resembles real-world data. Proposed by Goodfellow et al. [41], GANs usually comprise two neural networks, the generator and the discriminator, which are trained concurrently through an adversarial process.

The generator receives, as input, a random vector sampled from a latent space, typically defined by a normal or uniform distribution. This network transforms the input vector into data that resembles real-world data, represented by the training dataset [41–44]. The latent space, from which the random vector is drawn, is generally lower-dimensional compared to the space of the actual data. The goal of the generator is to learn how to map the different points composing the latent space to real data.

Conversely, the discriminator is a binary classifier that evaluates whether a sample is real (from the training dataset) or synthetic (crafted by the generator) [41–44]. The training is termed adversarial because the two networks contest each other in a zero-sum game, where the gain/success of the generator is the loss/failure of the discriminator and vice versa. Figure 2 depicts the training of a typical GAN [41–44].

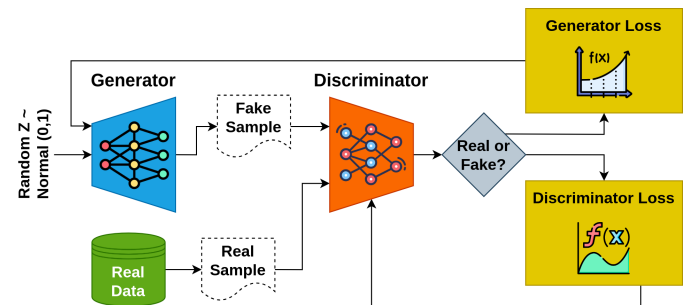


Fig. 2. GAN training: the parameters of the generator and discriminator are updated until the discriminator can not distinguish fake from real samples.

As observed in Figure 2, the generator does not access the training dataset. Instead, it learns by generating data and receiving feedback from the discriminator, which accesses original and synthetic samples. During training, both networks are optimized through backpropagation. The training ideally ends when the generator fools the discriminator half of the time, indicating that the discriminator can not reliably differentiate between real and synthetic data (50% accuracy) [41–44].

D. Related Work

In this section, we review and analyze the most significant works proposed to date for protecting the intellectual property and/or privacy of ML models deployed in untrusted devices. The discussion is structured according to the specific strategies employed by these works.

a) Full ANN on TEE: Bayerl et al. [8] propose to deploy the whole ANN at a non-secure memory region as cyphertext. As the ML system starts, the full ANN is loaded to the secure-world, where it is decrypted and stored in secure memory. The inference runs entirely within the secure-world. The framework proposed by Bayerl et al. [8] - Offline Model Guard (OMG) - targets Arm Cortex-A processors supporting SANCTUARY security architecture. Despite the simplistic approach, OMG does not scale well as the whole ANN has to fit within the secure memory, which tends to be very small.

The works proposed by Hanzlik et al. [1] and VanNostrand et al. [7] reduce the TEE memory footprint by deploying the ANN to the TEE layer-by-layer. As the ML system starts, the parameters of the first layer are loaded into the TEE, where they are decrypted and stored in a secure memory buffer. As the APU calculates the outputs of the first layer, the application repeats the previous process for the following layer till it reaches the last one. This strategy significantly reduces the memory footprint of the TEE, which only has to guarantee enough memory to save the largest layer instead of the whole ANN. These works differ in the target SoC ISA - while MLCapsule [1] targets Arm Cortex-A processors with support to OP-TEE, the work of VanNostrand et al. [7] target x86 processors with Intel-SGX. Nakai et al. [15] improved over MLCapsule by using the shared cache memory between the normal and secure worlds to perform parameter loading in parallel with the parameter decryption and inference, thereby reducing the decision latency.

Liu et al. [9] followed a disruptive approach and proposed a framework (SecDeep) to reformulate Arm-NN, an Application Programming Interface (API) for ML workloads in Arm Cortex-A processors. Their analysis revealed that over 90% of the Arm-NN code is dedicated to tasks such as tensor preparation and performance optimization, while less than 10% of the code is involved in tensor computations that handle private/sensitive model or user data. Building on this insight, SecDeep partitions the Arm-NN API into two categories: confidential and non-confidential functions. Confidential functions are executed in the secure-world, while non-confidential functions are assigned to the normal-world.

b) TEE Extension: Xie et al. [10] also target Arm Cortex-A APUs, but instead of adapting ML models or

libraries to fit within the constraints imposed by TEEs, they enhanced memory management within TEEs to support ANN inference more efficiently. Xie et al. [10] introduce a novel MMU (Smart-Zone) that dynamically adjusts the secure memory size (number of page tables in virtual memory and the secure memory region in physical memory) required by a given ANN. In addition, the authors propose Tinylib, a lightweight library that optimizes ML operations within the secure-world, while maintaining a small TCB.

Within the same line, Yang et al. [16] extended the Intel-SGX technology of x86 processors with an MMU tailored for ML computation. In addition, the authors propose a framework to decompose an ANN and set optimal parameter blocks for each layer, minimizing the workload within the TEE.

c) Model Refactoring: Liu et al. [20] introduced MirrorNet, a framework to refactor an ANN into two ANNs, one executed within the normal-world and another within the secure-world. The refactored ANNs follow the architecture of the original model, but the version running within the secure-world has smaller kernels while contributing the most to the final decision. By its side, the model deployed in the normal-world must not be accurate enough to enable an attacker to reconstruct the original model. The authors tested MirrorNet on an Arm Cortex-A processor featuring Arm TrustZone.

d) Weight Obfuscation: Hou et al. [2] propose to obfuscate a given percentage of weights in descent order of their magnitude, i.e. contribution to the final decision. With this design approach, the inference process runs almost entirely outside the TEE, being the enclave responsible to denoise the activations of each layer. For any input sample, the crafted noise must be enough to change the predicted label. The authors consider the ANN intellectual property as properly protected when a possible attacker has to pay the same machine training hours to recover the original model as the model owner training from scratch. Hou et al. [2] tested the concept on an x86 APU with Intel-SGX.

e) Selected Layers on TEE: Slalom [17] is among the first works to delegate only a restricted set of layers to the TEE. For this purpose, Slalom splits an ANN between linear and non-linear layers. The inference is managed from the enclave, where the non-linear layers are executed. For linear layers, Slalom encrypts the input data and layer parameters, outsourcing the computations to a co-located GPU. Once the GPU completes the computation of a linear layer, the activations are decrypted within the enclave, and their integrity is verified using Freivalds' algorithm. Slalom is designed for x86 processors equipped with Intel-SGX. Building on this concept, Sun et al. [18] proposed ShadowNet, a similar framework optimized for Arm Cortex-A processors that support OP-TEE.

The SecureFrameNet framework [19] builds upon model pruning techniques to establish a metric for quantifying the contribution of each neural network layer to the final decision. The importance score of each layer is then used to partition the ANN into two or more subgraphs. During the segmentation process, SecureFrameNet also evaluates the FLOPs associated with each subgraph. SecureFrameNet targets Arm Cortex-A processors with support to protected Virtual Machine (pKVM).

TABLE I
GAP ANALYSIS

	Target Device			Category	Trusted Execution Summary
	Architecture	TEE	TinyML		
VanNostrand et al. [7] 2019	Arm Cortex-A	OP-TEE	No	Full-ANN	Saves the ANN as cyphertext in non-secure memory. Loads the model layer-by-layer to the TEE.
OMG [8] 2020	Arm Cortex-A	SANCTUARY	No	Full-ANN	Saves the ANN as cyphertext in non-secure memory. Loads the model to the TEE in a single iteration.
MLCapsule [1] 2021	x86	Intel-SGX	No	Full-ANN	Saves the ANN as cyphertext in non-secure memory. Loads the model layer-by-layer to the TEE.
Nakai et al. [15] 2021	Arm Cortex-A	OP-TEE	No	Full-ANN	Uses the shared cache between the REE and the TEE to load the model in parallel with prediction.
SecDeep [9] 2021	Arm Cortex-A	OP-TEE	No	Full-ANN Conf. Functions	Arm-NN functions dealing with user and model data and are delegated to the TEE.
Xie et al. [10] 2024	Arm Cortex-A	Arm TrustZone	No	Full-ANN TEE Extension	Custom MMU/API for ML computation within TEE.
Yang et al. [16] 2024	x86	Intel-SGX	No	Full-ANN TEE Extension	Custom MMU for ML computation within TEE.
MirrorNet [20] 2023	Arm Cortex-A	Arm TrustZone	No	Model Refactoring	Refactors 1 ANN in 2 ANNs. The ANN within the TEE contributes the most to the classification task.
Hou et al. [2] 2022	x86	Intel-SGX	No	Weight Obfuscation	Obfuscates strategical weights and denoises the output of each layer within the TEE.
Slalom [17] 2019	x86	Intel-SGX	No	Selected Layers	Offloads the linear layers from the TEE to the GPU.
DarkneTZ [6] 2020	Arm Cortex-A	OP-TEE	No	Selected Layers	Offloads layers that potentiate MIAs to the TEE.
ShadowNet [18] 2023	Arm Cortex-A	OP-TEE	No	Selected Layers	Offloads the linear layers from the TEE to the GPU.
SecureFrameNet [19] 2024	Arm Cortex-A	N.A.	No	Selected Layers	Uses pruning-based techniques to identify the layers that should be offloaded to the TEE.
Costa et al. 2024	Arm Cortex-M	TF-M	Yes	Selected Layers	Offloads layers that lessen the effort of an attacker to build a surrogate QNN to the TEE.

Mo et al. [6] proposed a framework with a slightly off goal but still related to the intellectual property of ML models. Instead of preserving the privacy of ANN parameters like previous works, DarkneTZ seeks to protect the privacy of training data by offering increased robustness against MIAs. DarkneTZ traverses the model layer-by-layer, starting from the last layer, to determine which layers contribute the most to the success of a powerful white-box MIA. This set of layers is delegated to the secure-world. DarkneTZ targets Arm Cortex-A processors with support to OP-TEE.

E. Gap Analysis

Table I puts our work in perspective with the works previously described. As observed, previous research focuses on Arm Cortex-A processors with TrustZone and x86 processors supporting Intel-SGX. To the best of our knowledge, our framework is the first to account for the constraints imposed by Arm Cortex-M MCUs and TrustZone-M.

a) Full ANN on TEE: Deploying the ANN layer-by-layer to the secure-world [1, 7, 15] reduces the TEE memory footprint by requiring memory allocation only for the largest layer. However, this approach requires a cryptographic operation for each layer upon receiving an input sample. Arm

Cortex-M MCUs not only feature much lower computing power than Cortex-A and x86 processors but also lack support for GPU acceleration and shared caches between normal and secure worlds to compensate for the additional overhead.

SecDeep (designed for Arm Cortex-A) follows a disruptive approach, by delegating only to the secure-world the functions that deal with user or ANN data - which accounts for only 10% of the Arm-NN API. However, CMSIS-NN is already fully optimized and does not include functions for tensor preparation or hardware configuration like Arm-NN, making SecDeep not appropriate for Cortex-M MCUs.

b) TEE Extension: Extending the TEE via a custom MMU for ML computation [10, 16] is infeasible on Cortex-M MCUs, as these devices lack MMUs (Section II-A). While extending the TEE for ML workloads presents an avenue for future research, existing approaches for Cortex-A [10] and x86 processors [16] are not directly portable.

c) Model Refactoring: The model refactoring strategy proposed by Liu et al. [20] is hardly portable to devices powered by Arm Cortex-M MCUs as they are short in memory and do not feature more than one core to be able to store and run two ANNs in parallel.

d) Weight Obfuscation: The weight obfuscation technique proposed by Hou et al. [2] is among the most efficient to date. However, it still introduces computational overhead for denoising the activations/output of each ANN layer.

e) Selected Layers on TEE: Slalom [17] and ShadowNet [18] are not portable to Arm Cortex-M MCUs due to reliance on GPU acceleration. Nevertheless, storing and executing strategic layers on the TEE is a viable approach to increasing intellectual property protection of QNNs in Cortex-M MCUs without incurring computation overhead. To trade off TEE memory footprint and decision latency, our framework (SecureQNN) follows a strategy similar to SecureFrameNet [19] and DarkneTZ [6]. Nevertheless, while SecureFrameNet perceives the criticality of each layer based on the relevance to the final decision, our framework perceives layer criticality in terms of how it lessens the effort of a possible attacker in constructing a surrogate model with equal or higher accuracy. Additionally, SecureFrameNet relies on the number of FLOPs to manage the TEE computational load, a metric incompatible with the integer-precision execution typical of Cortex-M MCUs. DarkneTZ, in contrast, primarily addresses the privacy of the training data (MIAs) rather than ANN privacy.

IV. SECUREQNN

Figure 3 details the high-level architecture of SecureQNN, composed of three main modules executed in the following order: (i) Surrogate Dataset Generator, (ii) Model Partitioning, and (iii) Trusted Execution. The Surrogate Dataset Generator relies on deep generative models to simulate the creation of a surrogate dataset by a possible attacker. This dataset is then used as input for the Model Partitioning module, which simulates a series of training sessions performed by a powerful attacker to determine the privacy-critical layers. The resulting QNN partitioning setting is then used as input for the Trusted Execution module, which generates the C code that provisions an edge device powered by an Arm Cortex-M MCU with SecureQNN privacy features.

A. Surrogate Dataset Generator

A potential attacker lacks access to the original training dataset. Nevertheless, they can spoof sensors or network connections to steal inference data and apply data augmentation techniques to build a surrogate training dataset. We simulate this scenario using deep generative models for image generation upon the original test dataset.

a) Deep Generative Model Architecture: To determine the most suitable model architecture for our use case, we referenced a study conducted in 2022 [45], which benchmarked: (i) Generative Adversarial Networks (GANs), (ii) Energy-Based Models (EBMs), (iii) Variational Autoencoders, (iv) Autoregressive Models, and (v) Normalizing Flows. These models were benchmarked on the CIFAR-10 dataset, which we also use to evaluate our framework. Among others, the study reports the Fréchet Inception Distance (FID), a widely supported and adopted metric for the quantitative evaluation of deep generative models for image generation [44, 46].

Results indicate that GANs and EBMs perform notably better than the other architectures. To further refine our choice, we evaluated the availability of these models through a unified programming framework. Ultimately, we chose GANs because of the availability of StudioGAN [44], a Python library that supports experimentation with state-of-the-art GAN architectures using both default and custom datasets. The authors of StudioGAN also provide a comprehensive benchmark for different GAN architectures and configurations on different datasets and even propose a training protocol based on empirical experimentation. To minimize the engineering effort in the development of the Surrogate Dataset Generator, we relied on the benchmark tables provided by the authors of StudioGAN and followed their training protocol.

b) GAN Evaluation Backbone: The evaluation backbone is a pre-trained ANN that processes real images (from the training dataset) and synthetic images (generated by the GAN). Its purpose is to extract a feature set used to compute metrics that quantify the quality of the generated images, such as the FID. StudioGAN supports and benchmarks three evaluation backbones: (i) TensorFlow-InceptionV3, (ii) PyTorch-SwAV, and (iii) PyTorch-Swin-T. To identify the most suitable evaluation backbone for our use case, we focused our analysis on the FID scores obtained for the CIFAR-10 and ImageNet datasets. StudioGAN's benchmark indicates that PyTorch-SwAV holds the best results.

c) GAN Architecture - CIFAR-10: For CIFAR-10, we have to select a technique for data-efficient training because the attacker only has access to a small dataset (the CIFAR-10 test dataset only contains 10k samples). Without data-efficient training, the discriminator memorizes the training dataset, making the training collapse. From the GAN architectures benchmarked in StudioGAN, we only considered options with data augmentation or LeCam regularization - the two data-

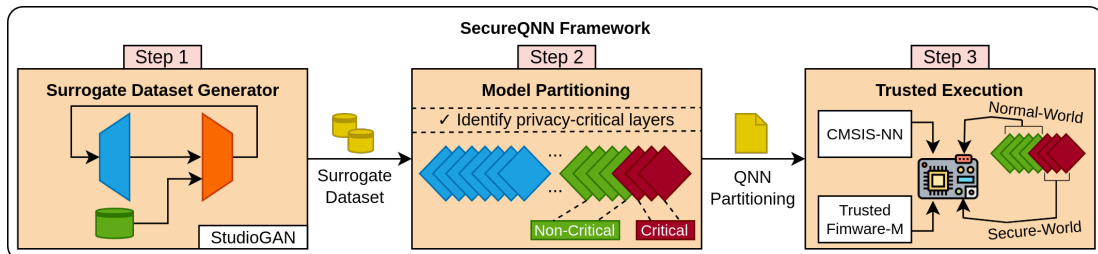


Fig. 3. High-level architecture of SecureQNN. The Surrogate Dataset Generator simulates the generation of a training dataset by a potential attacker. The Model Partitioning divides the QNN into critical and non-critical layers. The Trusted Execution deploys the C code on the target MCU.

efficient training techniques supported by StudioGAN. The last design parameter to consider is the training backbone, i.e. the architecture of the generator and discriminator (GAN architecture). For CIFAR-10, StudioGAN benchmarks two architectures coupled with data-efficient training: (i) StyleGAN and (ii) BigGAN. To maximize the diversity of the surrogate dataset, we selected one GAN from each training backbone. The results detailed in Section V-A1 led us to integrate a StyleGAN2 and a standard BigGAN in our framework, both coupled with Differential Augmentation. The former returned an FID of 0.91 and the latter returned an FID of 1.59.

d) GAN Architecture - VWW: Since StudioGAN does not benchmark GANs tailored for the VWW dataset or its resolution (96x96), we adapted configurations from other datasets and applied an appropriate filter for image resizing. Among the image resolutions supported by StudioGAN, 128x128 is the closest to the original VWW resolution, being 1.33x higher. This is the resolution of the ImageNet dataset, whose GAN configurations were used as a basis for our implementation on VWW. For image resizing, we applied a bicubic filter, as advised in the training protocol of StudioGAN. StudioGAN benchmarks two GAN architectures on ImageNet: (i) BigGAN and (ii) StyleGAN. Nevertheless, we ended up including two variations of StyleGAN (StyleGAN2 and StyleGAN3-t) in our framework as we were not able to get good quality images from BigGAN models (Section V-A2). StyleGAN2 returned an FID of 4.17, while StyleGAN3-t returned an FID of 7.05.

B. Model Partitioning

The layers of an ANN do not contribute equally to the classification task [47–49]. While early layers contribute the most to decomposing the input data, later layers contribute the most to the final prediction. Building on this insight, we design a method to partition a given QNN into secure and normal worlds, aiming to achieve a good trade-off between TEE memory footprint and inference latency. For this purpose, we propose to divide the QNN into two blocks only - one to

be stored and executed within the normal-world and another to be stored and executed within the secure-world.

Saving the whole QNN in non-secure memory, using cryptography to protect the privacy of critical layers, similarly to previous works, is the best strategy to reduce the memory footprint of the secure-world. Under this strategy, the secure-world only has to feature enough memory for the parameters of one layer at a time. However, this strategy does not port well to Arm Cortex-M MCUs as cryptography operations are too computationally expensive compared to Arm Cortex-A and x86 processors. This aligned with the increased context switches jeopardizes the inference latency. Nevertheless, by giving preference to deploy later layers to the secure-world instead of early ones, we are still able to significantly reduce the TEE memory footprint while also protecting the most critical layers to the classification task [47–49]. Later layers tend to feature significantly smaller activation vectors, which inherently reduces the size of intermediate buffers between layers. Figure 4 depicts the workflow of the Model Partitioning module. This module is developed upon the TensorFlow v2 API as it enables a seamless conversion of floating-point ANNs to integer precision QNNs compatible with TensorFlow Lite Micro and CMSIS-NN - the state-of-the-art APIs for ML computation on Arm Cortex-M.

Our framework analyzes the QNN layer-by-layer, starting from the final layer and moving backward. For each new layer, our framework performs a training process where the parameters of the new layer and the forwarding ones are set as trainable and updated at each epoch, while the parameters of the other layers are frozen (non-trainable). For training the surrogate QNN, our framework employs a dataset merging the original test dataset with the synthetic datasets produced by the two GANs. The training ends under one of two conditions: (i) when the accuracy of the surrogate QNN reaches or exceeds that of the target QNN, or (ii) when the number of training epochs matches that of the target QNN, but the accuracy remains lower. In the first scenario, the framework sets one more layer as trainable and repeats the training process. Every

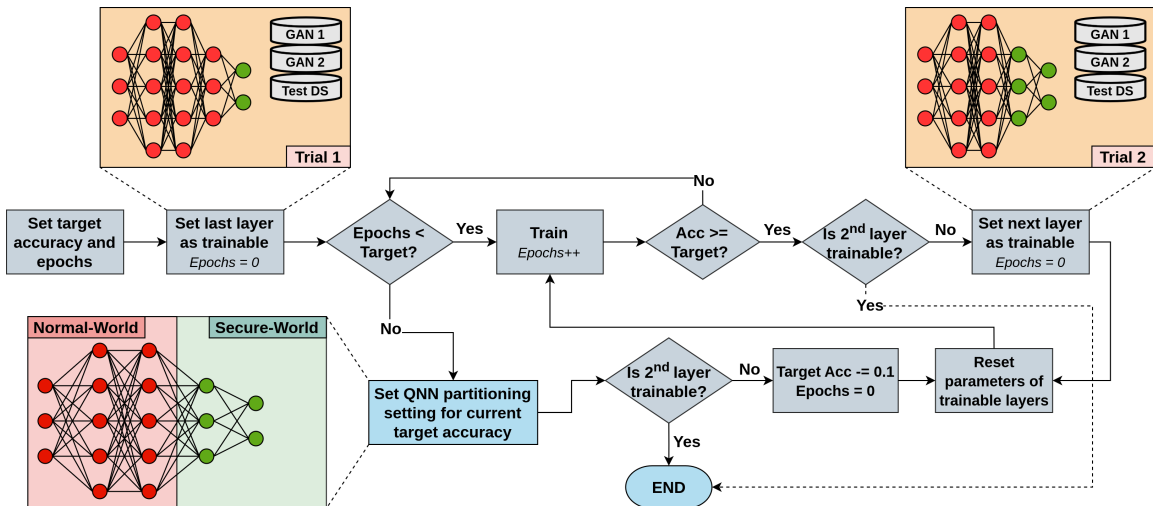


Fig. 4. Workflow of the Model Partitioning module. It stops adding layers to the secure-world when the accuracy of the surrogate QNN fails to reach the target within the same training epochs as the original QNN. Red highlights layers for the normal-world, while green highlights layers for the secure-world.

time the framework sets one more layer as trainable, the parameters of all trainable layers are reinitialized to a truncated normal distribution to avoid overfitting. The second scenario means that the framework has found a QNN partitioning setting that requires the attacker to pay the same effort to reconstruct the QNN as the SP training from scratch. The final set of trainable layers are the ones that must be delegated to the secure-world. We assume a worst-case scenario where the adversary knows the architecture of the whole QNN, including the private layers, and training hyperparameters.

The scenario where SecureQNN delegates to the secure-world the minimum set of layers to undermine the cost-benefit of unauthorized QNN replication is termed as the default privacy setting. Under this scenario, our framework sets the accuracy of the surrogate QNN to just below the accuracy of the original QNN. After determining the set of layers to comply with the default privacy setting, our framework reduces the target accuracy of the surrogate QNN in buckets (10% by default), repeating the process previously described (Figure 4). SecureQNN keeps reducing the target accuracy of the surrogate QNN until all layers but the first one is considered known to the attacker. This enables the SP to trade-off between increased privacy and memory allocated to the secure-world.

C. Trusted Execution

Figure 5 depicts the architecture of an ML system provisioned by SecureQNN. As illustrated, featuring Arm TrustZone-M and supporting TF-M are the minimum requirements for an MCU to support our framework. TrustZone-M establishes two separate execution environments, with each environment supporting privileged and non-privileged execution levels. The privileged-level blocks are colored in gray, while the non-privileged blocks are color-coded as green for the secure-world and red for the normal-world. TF-M is used as the reference implementation of the secure firmware.

a) Secure Boot: Within the privileged level of the secure-world, Secure Boot is mandatory as it verifies the digital signature of the firmware image during the device startup. Anytime the signature is corrupted, the boot is halted, preventing the execution of malicious code. Therefore, Secure Boot is essential to detect any tampering attempt of the QNN along its private and public layers. As a reference for the implementation of a secure boot, TF-M uses MCUboot.

b) Firmware Update: Also in the privileged level of the secure-world, Firmware Update is an optional service of

TF-M useful if the SP updates the QNN over-the-air. This service ensures that whenever new firmware is downloaded to the device, it is only installed if its digital signature is valid. Additionally, Firmware Update includes an anti-rollback mechanism to prevent attackers from installing an older, potentially vulnerable version of the firmware and QNN.

c) Secure-World (SW): In the non-privileged level of the secure-world, SecureQNN relies on two primary Trusted Applications (TAs). Crypto is an optional service implemented by TF-M and provides cryptographic functionalities, including symmetric/asymmetric decryption, which enables our framework to tackle flash probing attacks tailored at stealing the parameters of private layers. If turned on, the parameters (weights and bias) of private layers are saved in a secure region of the flash memory as ciphertext. During inference, the Crypto service decrypts the parameters layer-by-layer as required. The decrypted parameters are stored in a single SRAM buffer, which is erased as the parameters are no longer needed for inference on the current sample. We store the parameters of private layers in secure memory even when TF-M Crypto is active to reduce the number of context switches and not jeopardize the inference latency. The Private QNN TA computes the output of the private layers. Whenever this TA is called, it loads the parameters directly from the flash, if they are stored in plain text, or interacts with the TF-M Crypto service to decrypt them, if in ciphertext. The final output is sent in plain text to the Public QNN Client Application (CA).

d) Normal-World (NW): In the normal-world, most software components are optional, except for the Input Sensors and Public QNN CAs. The former collects and processes the raw data from sensors, so it can be used as input for the QNN. The latter initiates and manages the inference process until a private layer is found. Whenever a private layer is found, the inference process is transferred to the Private QNN TA.

V. EVALUATION

To evaluate our framework under conditions that resemble a realistic scenario, we relied on the MLPerf Tiny benchmark [25]. This benchmark features QNNs and datasets that reflect the use cases described by more than 30 entities from industry and academia for TinyML systems. Such use cases include scenarios demanding real-time decisions and others where this requirement can be relaxed. Our evaluation focused exclusively on image applications, as images offer a practical means to assess the quality of the surrogate dataset generated by an attacker. Regarding image processing applications, MLPerf Tiny comprises two QNNs: (i) ResNet trained on CIFAR-10 and (ii) MobileNetV1 trained on VWW. CIFAR-10 contains 32x32 images distributed over 10 classes, each representing a different object. VWW is a binary dataset of 96x96 images that represent the use case of detecting whether a person is present in an image. The weights of each QNN are quantized to int-8 precision, while the bias is to int-32.

A. Surrogate Dataset

Table II reports the FID scores of the GANs we trained for the CIFAR-10 and VWW datasets. The FID metric quantifies

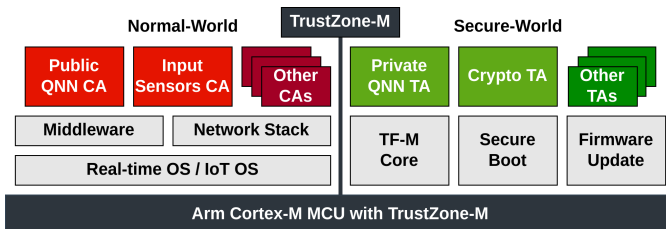


Fig. 5. Architecture of an ML system provisioned by SecureQNN. Gray highlights privileged-level software. Green/red highlights non-privileged software within secure/normal worlds.

TABLE II

EVALUATION OF GANS FOR SURROGATE DATASET GENERATION. GANS INTEGRATING SECUREQNN ARE HIGHLIGHTED IN GREEN. FID MEASURES THE SIMILARITY BETWEEN SURROGATE AND REAL DATASETS.

Dataset	GAN	Input Dim	FID		
			[0, +∞]	Surrogate	Original
CIFAR-10	BigGAN+LeCam	32x32	2.26	-	
	BigGAN+DiffAug	32x32	1.59	88.38%	86.53%
	StyleGAN2+DiffAug	32x32	0.91	85.00%	
VWW	ReACGAN-256	128x128	11.17	-	
	BigGAN-256	128x128	13.68	-	
	StyleGAN2	128x128	4.17	81.77%	85.75%
	StyleGAN3-t	128x128	7.05	83.21%	

the disparity between two sets of images - real and synthetic - by comparing the distributions of features extracted from an intermediate layer of a pre-trained network (evaluation backbone). Both real and synthetic images are modeled as Gaussian distributions, characterized by their respective means and covariance matrices. The FID score is computed based on the difference between these distributions. FID scores range from 0 to $+\infty$, where a lower score corresponds to a higher image similarity - more realistic synthetic images.

The GAN configurations selected to be part of our framework are highlighted in green in Table II. For these configurations, we also measured the accuracy of the target QNN when tested on synthetic images and compared it to the baseline test accuracy, i.e., the accuracy of the original QNN measured on the original test dataset.

1) *CIFAR-10*: To maximize the diversity of the surrogate dataset, we evaluated variations of two distinct GAN architectures: (i) StyleGAN and (ii) BigGAN (Section IV-A). Regarding the StyleGAN architecture, StyleGAN2 with Differential Augmentation (StyleGAN2 + DiffAug) benchmarks the best/lowest FID score within StudioGAN and we were able to get good quality images from it (FID=0.91). Regarding the BigGAN architecture, BigGAN benchmarks the best FID score in StudioGAN when coupled with LeCam regularization (BigGAN + LeCam). Unfortunately, for our use case, LeCam regularization was insufficient to avoid overfitting. As shown in Figure 6, the lowest FID registered in 105k steps is 2.26 (vs. 0.45 on benchmark) and remained the lowest value after 50k steps. Nevertheless, replacing LeCam regularization with Differential Augmentation (BigGAN + DiffAug) was sufficient to get good quality images (FID=1.59).

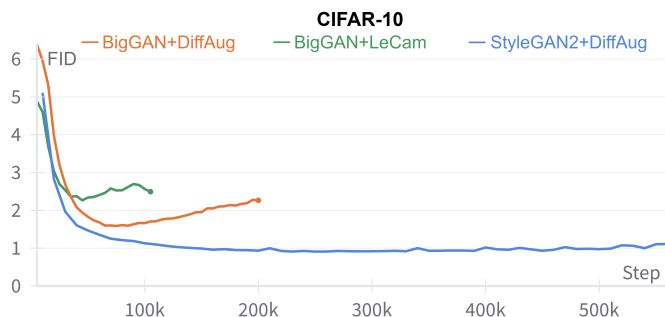


Fig. 6. FID evolution over training steps for CIFAR-10 GANs.

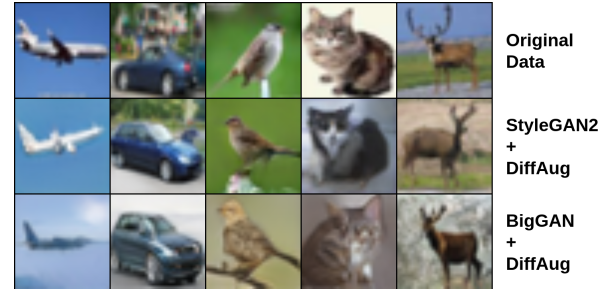


Fig. 7. CIFAR-10 samples (line 1) vs. synthetic GAN samples (lines 2-3).

Figure 7 puts images crafted by the selected GANs in perspective against the original training data. Although we were not able to reach FID scores as low as the ones benchmarked in StudioGAN - we train GANs on the original test dataset to simulate a realistic attacking scenario - we could still get high quality images. For synthesis purposes, we only show images of five out of the ten classes composing the CIFAR-10 dataset. As observed, figures of airplanes, cars, birds, cats, and deer are completely discernible in the synthetic images. Furthermore, we also observe that the selected GANs were able to generate these figures with different backgrounds, increasing the diversity of the dataset. The high quality of synthetic/surrogate datasets is reflected in the accuracy of the target QNN when tested on these synthetic data. The maximum accuracy loss is registered for the StyleGAN2 architecture; however, the loss remains low at 1.53%.

2) *Visual Wake Words (VWW)*: For the VWW dataset, we also evaluated variations of different GAN architectures (Section IV-A). Regarding BigGAN, the ReACGAN-256 and the BigGAN-256 configurations benchmark the best/lowest FID scores on ImageNet (the dataset with the closest image resolution to VWW). However, when applied to the VWW dataset, both configurations result in training collapse, with the FID scores stagnating at 11.17 and 13.68, respectively. As illustrated in Figure 8, these FID scores are observed in the initial training steps - step 30k for ReACGAN-256 and step 8k for BigGAN-256 - after which the FID increases rapidly without ever converging to lower values.

In contrast, the StyleGAN architecture, namely the StyleGAN2 and StyleGAN3-t configurations (the only ones benchmarked in StudioGAN) yielded good results on the VWW dataset. As shown in Figure 8, these configurations do not

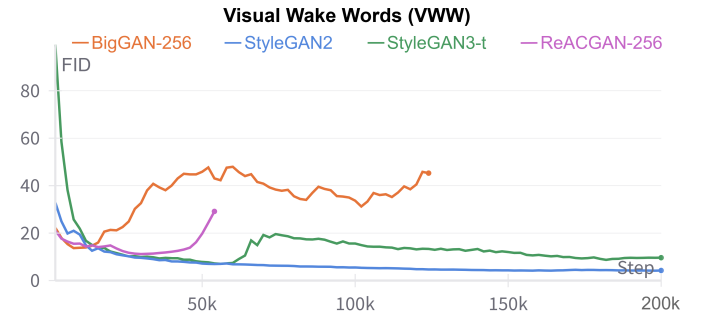


Fig. 8. FID evolution over training steps for VWW GANs.



Fig. 9. VWW samples (line 1) vs. synthetic GAN samples (lines 2-3).

make the training collapse. The StyleGAN2 and StyleGAN3-t configurations return an FID score of 4.17 and 7.05, respectively, and we could get good quality images from them as shown in Figure 9. For synthesis purposes, we only show images representing the scenario/class where a human is present in the image. Although there is room for improvement, human-like figures are discernible in the synthetic images, which is reflected in the accuracy of the target QNN when tested on this synthetic data. As detailed in Table II, the accuracy is comparable to the obtained with the original test data, with a maximum accuracy loss of 4.11% registered for the StyleGAN2 configuration.

As illustrated in Figure 8, training was halted at 200k steps after no further improvement in FID score was observed for over 50k consecutive steps. Due to hardware constraints, extending the training of VWW GANs for marginal FID improvements, as was done for the CIFAR-10 dataset (Figure 6), was not feasible. Unlike CIFAR-10 GANs, which process 32×32 images, VWW GANs operate on 128×128 images, significantly increasing resource demands during training.

B. Privacy vs. Memory Trade-Off

Table III reports the minimum memory required to store QNN parameters within the Private QNN TA (Figure 4) as a function of the maximum accuracy that an adversary might achieve when training a surrogate QNN. The table reflects the worst-case scenario out of four trials, specifically reporting the trial in which the highest number of layers needed to be transferred to the secure-world.

TABLE III
MINIMUM TEE MEMORY FOOTPRINT VS. MAXIMUM ACCURACY OF SURROGATE QNN

QNN	Max. Accuracy Surrogate QNN	# Private Layers	TEE Memory Footprint % QNN size in TEE
ResNet CIFAR-10	86.53%	3 / 10	39.16 KB / 76.90 KB (50.92%)
	76.53%	5 / 10	58.04 KB / 76.90 KB (75.47%)
	66.53%	7 / 10	71.79 KB / 76.90 KB (93.36%)
MobileNetV1 VWW	86.12%	9 / 28	137.38 KB / 213.93 KB (64.22%)
	76.12%	23 / 28	212.57 KB / 213.93 KB (99.36%)

As detailed in Table III, it is feasible to preserve the intellectual property of a QNN, to the point of undermining the cost-benefit of unauthorized replication, without transferring the execution of the whole model to the secure-world. The privacy of the CIFAR-10 ResNet can be maintained by transferring only the last 3 out of 10 layers to the secure-world, which corresponds to 50.92% of the total model size. In the case of MobileNetV1 trained on the VWW dataset, this requirement increases to 9 out of 28 layers, representing 65.27% of the total model size.

After determining the minimum set of layers to comply with the default privacy setting - the minimum set of layers that prevent an attacker from reconstructing a surrogate QNN with the same accuracy as the original model without expending more effort than the SP - SecureQNN starts to decrease the target accuracy achievable by an attacker in buckets of 10%. This enables the SP to trade off increased privacy and TEE memory footprint. As detailed in Table III, for the CIFAR-10 ResNet, it is possible to decrease the target accuracy by, at least, 20% before being necessary to save and execute the whole QNN on the secure-world. For the VWW MobileNetV1, this value reduces to 10%.

The memory values reported in Table III account only for the internal parameters of the QNN (weights and bias). In addition, the MCU where the QNN is deployed must feature enough flash and SRAM to execute the CMSIS-NN library and the TF-M firmware, along with any necessary services. To better understand the memory footprint of the TEE in a real-world scenario, we developed an application based on TF-M v1.7 to perform inference using the CIFAR-10 and VWW QNNs. Results were obtained under the default privacy settings for QNNs and for configurations where the TF-M Crypto service was both enabled and disabled. Table IV contextualizes the TEE memory footprint of this minimalist application relative to the total SRAM and flash memory capacities of several development boards supported by TF-M v1.7. With TF-M Crypto disabled, the use of SecureQNN is feasible even on very low-budget platforms such as the STM Nucleo-L552ZE-Q. On this development board, the highest TEE memory usage is observed in SRAM when running the VWW QNN; however, the usage remains relatively low at 38.46%. When TF-M Crypto is enabled, it may be more appropriate to consider more expensive and resource-rich platforms, such as the STM B-U585I-IOT02A or the Nordic NRF5340-DK, particularly when working with larger QNNs like the VWW MobileNetV1.

C. Latency

We compared the decision latency per input sample imposed by SecureQNN against the baseline reference scenario where the full QNN is executed within the normal-world. The experiments were carried out considering the default privacy setting of SecureQNN and using TF-M v1.7 on an STM Nucleo-L552ZE-Q, which features an Arm Cortex-M33 at 110 MHz. Results are detailed in Table V.

SecureQNN imposes two context switches between the normal and secure worlds per sample, which might suggest an increased decision latency compared to the baseline reference. However, the results reported in Table V show that

TABLE IV

TEE MEMORY FOOTPRINT OF A MINIMALIST APPLICATION BUILT UPON TF-M V1.7. THE APPLICATION WITHIN THE SECURE-WORLD ONLY STORES AND EXECUTES THE SET OF PRIVATE LAYERS COMPLYING WITH THE DEFAULT PRIVACY SETTING. THE APPLICATION WAS COMPILED TO REDUCE THE MEMORY SIZE OF THE FIRMWARE (-Os).

Board	Memory Size (KBytes)		CIFAR-10		VWW		CIFAR-10 (Crypto)		VWW (Crypto)	
	SRAM	Flash	SRAM %	Flash %	SRAM %	Flash %	SRAM %	Flash %	SRAM %	Flash %
Musca-B1	512	2048+8192	5.36%	0.78%	5.15%	1.91%	26.04%	1.76%	31.44%	2.89%
Musca-S1	512	2048+32768	5.36%	0.23%	5.15%	0.56%	26.03%	0.52%	31.44%	0.85%
nRF5340 DK	512	1024	5.32%	8.15%	5.10%	20.71%	26.17%	18.20%	31.57%	29.20%
nRF9160 DK	256	1024	10.64%	9.70%	10.21%	19.14%	52.34%	19.75%	63.14%	29.19%
LPCXpresso55S69	320	640	8.62%	11.53%	8.28%	29.58%	42.93%	26.94%	51.57%	45.00%
B-U5851-IOT02A	786+8192	2048+65536	0.31%	0.13%	0.30%	0.31%	1.50%	0.28%	1.81%	0.45%
STM32L562E-DK	256	512+65536	10.67%	0.12%	10.24%	0.30%	52.39%	0.27%	63.20%	0.44%
NUCLEO-L552ZE-Q	256	512	10.67%	15.90%	10.24%	38.46%	52.39%	34.92%	63.20%	57.50%

TABLE V

DECISION LATENCY PER SAMPLE FOR DIFFERENT SECURITY SETTINGS. "NW" STANDS FOR NORMAL-WORLD, "SW" FOR SECURE-WORLD, AND "N.A." FOR NOT APPLICABLE.

QNN	Setting	# Private Layers	Inference CPU Cycles / Sample Context Switch NW->SW->NW	Latency/Sample
ResNet CIFAR-10	Normal World	3 / 10	111.45M 1.01s	N.A.
	NW+SW Crypto OFF	3 / 10	111.44M 1.01s	44.86K 0.41ms
	NW+SW Crypto ON	3 / 10	120.58M 1.10s	
MobileNetV1 VWW	Normal World	9 / 28	88.16M 0.80s	N.A.
	NW+SW Crypto OFF	9 / 28	88.04M 0.80s	47.91K 0.44ms
	NW+SW Crypto ON	9 / 28	120.80M 1.10s	

the time spent on context switches, including the transfer of the activations of the last public layer to the secure-world, is residual. When TF-M crypto is off, the two context switches account only for 0.04% and 0.06% of the total decision latency (in terms of clock cycles) in CIFAR-10 and VWW QNNs, respectively. If TF-M crypto is on, context switches represent 0.04% of the decision latency in both neural networks.

When we compare the decision latency imposed by Secure-QNN without TF-M crypto against the baseline reference, we observe a slight reduction of clock cycles per inference despite the additional clock cycles spent on the two context switches. We investigated this surprising result, and we concluded that this is related to compiler optimizations gained by splitting the execution of both QNNs into two parts. We can not guarantee that this clock cycle reduction will extend to other QNNs as compiler optimizations are pretty dependent on the target application. As memory is a critical resource in our testing platform, our tests were compiled using the *Os* optimization level. Although it reduces the firmware size, this optimization level might result in unaligned memory accesses which inherently increases the latency.

VI. DISCUSSION

Although shifting ML computation to the deep edge is an ongoing trend, preserving the intellectual property of ML models remains an open research topic. In this paper, we proposed the first public and open-source framework to increase intellectual property protection of QNNs in devices powered by Arm Cortex-M MCUs and proved its feasibility. However, there is still room for improvement.

a) **Trusted Firmware-M (TF-M) Limitations:** In TF-M, memory (SRAM and flash) for the secure and normal worlds is allocated during the boot process and remains statically partitioned thereafter. This approach presents no significant drawbacks in applications where the QNN is consistently or almost always active. However, in scenarios where QNNs are not required to run continuously, static memory allocation imposes unnecessary limits on the amount of software that can be deployed in the normal-world. Enhancing TF-M with dynamic memory allocation for the secure-world post-boot would be beneficial for such a use case. As detailed in Section III-A, the SAU unit of Arm TrustZone-M allows dynamic memory partitioning and is programmable.

b) **Enhanced Defense Against MIAs:** By protecting the privacy of critical layers, SecureQNN serves as a defense against white-box MIAs, designed to determine whether a specific sample is part of the training dataset. Although white-box MIAs are significantly more accurate than black-box MIAs, the latter still pose a threat that our framework does not mitigate. Integrating SecureQNN with established defenses against black-box MIAs could offer enhanced security guarantees for the SP. Among the defense mechanisms available for mitigating MIAs [50], defenses applied in the pre-training and training phases are more adequate than defenses in the inference phase. Pre-training and training defenses imply the retraining of the QNN; however, these defenses do not introduce overhead during inference - a key advantage given the resource-constrained nature of Arm Cortex-M MCUs.

c) **Extension to Other ML Architectures:** We evaluated our framework on ML models borrowed from the state-of-the-art benchmark for TinyML. The QNNs and datasets included in MLPerf Tiny are representative of applications identified by over 30 entities from both industry and academia. Notably, in addition to two image classification models, the benchmark

also includes two models for audio classification, namely for keyword spotting and anomaly detection. This suggests that extending SecureQNN to support audio-based ML represents a meaningful direction for future research. The audio models in MLPerf Tiny - a Depthwise Separable Convolutional Neural Network trained on Google Speech Commands and a Fully-Connected Autoencoder trained on ToyADMOS - are based on convolutional and fully-connected layers. Therefore, supporting such models within SecureQNN would primarily involve enhancing the Surrogate Dataset Generator to enable the synthesis of short audio clips. Generative models based on WaveGAN [51], which has demonstrated the capability to produce word-like audio and is adaptable to other domains, could be a good starting point. However, it is important to note that the QNNs in MLPerf Tiny do not operate on raw audio but on audio-derived features such as log-mel spectrograms or mel-frequency cepstral coefficients. Therefore, any audio data generation pipeline would need to include a suitable feature extraction component to bridge this gap.

Additionally, within the image processing domain, it would be worthwhile to explore other model architectures already implemented on TinyML hardware, such as Capsule Networks (CapsNets) [52]. Supporting CapsNets would require extending the Model Partitioning module to train capsule layers and updating the Trusted Execution module with the API proposed by Costa et al. [52].

VII. CONCLUSION

In this paper, we present SecureQNN, the first public framework to increase the intellectual property protection of ML models in TinyML systems powered by Arm Cortex-M MCUs featuring TrustZone-M technology. Results extracted for QNNs trained on CIFAR-10 and VWW datasets suggest that it is possible to undermine unauthorized QNN replication without delegating the execution of the whole model to the secure-world. Specifically, the default privacy setting was achieved by delegating the execution of only 3 out of 10 layers and 9 out of 28 layers to the secure-world, respectively. This corresponds to 51% to 65% of the total QNN size. Regarding latency, our framework only requires two context switches between normal and secure worlds per input sample, which corresponds only to 0.06% of the decision latency in the worst-case scenario. We are opening the artifacts in the hope of enabling independent validation of the results and encouraging further research on the privacy of QNNs in TinyML systems.

ACKNOWLEDGMENTS

This work has been supported by FCT - Fundação para a Ciência e Tecnologia within the R&D Unit Project of ALGORITMI Centre. Miguel Costa was supported by FCT grant SFRH/BD/146780/2019.

REFERENCES

- [1] Lucjan Hanzlik et al. "MLCapsule: Guarded Offline Deployment of Machine Learning as a Service". In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. 2021.
- [2] Jiahui Hou et al. "Model Protection: Real-Time Privacy-Preserving Inference Service for Model Privacy at the Edge". In: *IEEE Transactions on Dependable and Secure Computing* (2022).
- [3] Ion Stoica et al. "A Berkeley View of Systems Challenges for AI". In: *CoRR* (2017).
- [4] M. G. Sarwar Murshed et al. "Machine Learning at the Network Edge: A Survey". In: *ACM Comput. Surv.* (2021).
- [5] Diogo Costa et al. "Train Me If You Can: Decentralized Learning on the Deep Edge". In: *Appl. Sci.* (2022).
- [6] Fan Mo et al. "DarkneTZ: Towards Model Privacy at the Edge Using Trusted Execution Environments". In: *International Conference on Mobile Systems, Applications, and Services*. 2020.
- [7] Peter M. VanNostrand et al. "Confidential Deep Learning: Executing Proprietary Models on Untrusted Devices". In: *CoRR* (2019).
- [8] Sebastian P. Bayerl et al. "Offline Model Guard: Secure and Private ML on Mobile Devices". In: *Conference on Design, Automation and Test in Europe*. 2020.
- [9] Renju Liu et al. "SecDeep: Secure and Performant On-Device Deep Learning Inference Framework for Mobile and IoT Devices". In: *International Conference on Internet-of-Things Design and Implementation*. 2021.
- [10] Xueshuo Xie et al. "Memory-Efficient and Secure DNN Inference on TrustZone-enabled Consumer IoT Devices". In: *IEEE Conference on Computer Communications*. 2024.
- [11] Maria Rigaki et al. "A Survey of Privacy Attacks in Machine Learning". In: *ACM Comput. Surv.* (2023).
- [12] R. Shokri et al. "Membership Inference Attacks Against Machine Learning Models". In: *IEEE Symposium on Security and Privacy (SP)*. 2017.
- [13] Hongsheng Hu et al. "Membership Inference Attacks on Machine Learning: A Survey". In: *ACM Comput. Surv.* (2022).
- [14] Daniel Oliveira et al. "uTango: An Open-Source TEE for IoT Devices". In: *IEEE Access* (2022).
- [15] Tsunato Nakai et al. "Towards Trained Model Confidentiality and Integrity Using Trusted Execution Environments". In: *Applied Cryptography and Network Security Workshops*. 2021.
- [16] Mengda Yang et al. "Penetralium: Privacy-preserving and memory-efficient neural network inference at the edge". In: *Future Generation Computer Systems* (2024).
- [17] Florian Tramer et al. "Slalom: Fast, Verifiable and Private Execution of Neural Networks in Trusted Hardware". In: *International Conference on Learning Representations*. 2019.
- [18] Z. Sun et al. "ShadowNet: A Secure and Efficient On-device Model Inference System for Convolutional Neural Networks". In: *IEEE Symposium on Security and Privacy (SP)*. 2023.
- [19] Renju C Nair et al. "SecureFrameNet: A rich computing capable secure framework for deploying neural network models on edge protected system." In: *International Conference on AI-ML Systems*. 2024.

- [20] Ziyu Liu et al. "MirrorNet: A TEE-Friendly Framework for Secure On-Device DNN Inference". In: *IEEE/ACM International Conference on Computer Aided Design*. 2023.
- [21] Liangzhen Lai et al. "CMSIS-NN: Efficient Neural Network Kernels for Arm Cortex-M CPUs". In: *CoRR* (2018).
- [22] Robert David et al. "TensorFlow Lite Micro: Embedded Machine Learning for TinyML Systems". In: *Machine Learning and Systems*. 2021.
- [23] Ioan Lucan Orășan et al. "A Brief Review of Deep Neural Network Implementations for ARM Cortex-M Processor". In: *Electronics* (2022).
- [24] Miguel Costa et al. "David and Goliath: An Empirical Evaluation of Attacks and Defenses for QNNs at the Deep Edge". In: *IEEE European Symposium on Security and Privacy (EuroS&P)*. 2024.
- [25] Colby R. Banbury et al. "MLPerf Tiny Benchmark". In: *CoRR* (2021).
- [26] Miguel Costa et al. "SecureQNN: Introducing a Privacy-Preserving Framework for QNNs at the Deep Edge". In: *Data Science and Artificial Intelligence*. 2023.
- [27] Cristiano Rodrigues et al. "BUSTed!!! Microarchitectural Side-Channel Attacks on the MCU Bus Interconnect". In: *IEEE Symposium on Security and Privacy (SP)*. 2024.
- [28] Sandro Pinto et al. "Demystifying Arm TrustZone: A Comprehensive Survey". In: *ACM Comput. Surv.* (2019).
- [29] Xi Tan et al. "SoK: Where's the "up"?! A Comprehensive (bottom-up) Study on the Security of Arm Cortex-M Systems". In: *USENIX WOOT Conference on Offensive Technologies*. 2024.
- [30] *Global Microcontroller Market Size By Product (8-Bit, 16-Bit, 32-Bit), By Application (Automobile, Communication, Healthcare), By Geographic Scope And Forecast*. Tech. rep. Verified Market Research, 2024.
- [31] *TinyML: The Next Big Opportunity in Tech*. Tech. rep. ABI Research, 2021.
- [32] *TinyML: A Market Update*. Tech. rep. ABI Research, 2022.
- [33] Eric Flamand et al. "GAP-8: A RISC-V SoC for AI at the Edge of the IoT". In: *IEEE International Conference on Application-specific Systems, Architectures and Processors (ASAP)*. 2018.
- [34] *GAP9 processor — GreenWaves Technologies*. https://greenwaves-technologies.com/gap9_processor/. [Accessed 04-06-2024].
- [35] Angelo Garofalo et al. "PULP-NN: A Computing Library for Quantized Neural Network inference at the edge on RISC-V Based Parallel Ultra Low Power Clusters". In: *IEEE International Conference on Electronics, Circuits and Systems (ICECS)*. 2019.
- [36] MohammadHossein AskariHemmat et al. "Quark: An Integer RISC-V Vector Processor for Sub-Byte Quantized DNN Inference". In: *IEEE International Symposium on Circuits and Systems (ISCAS)*. 2023.
- [37] Ingo Hoyer et al. "Design of Hardware Accelerators for Optimized and Quantized Neural Networks to Detect Atrial Fibrillation in Patch ECG Device with RISC-V". In: *Sensors* (2023).
- [38] Lachit Dutta et al. "TinyML Meets IoT: A Comprehensive Survey". In: *Internet of Things* (2021).
- [39] Partha Pratim Ray. "A review on TinyML: State-of-the-art and prospects". In: *Journal of King Saud University - Computer and Information Sciences* (2022).
- [40] Youssef Abadade et al. "A Comprehensive Survey on TinyML". In: *IEEE Access* (2023).
- [41] Ian Goodfellow et al. "Generative Adversarial Nets". In: *Adv. in Neural Information Processing Systems*. 2014.
- [42] Antonia Creswell et al. "Generative Adversarial Networks: An Overview". In: *IEEE Signal Processing Magazine* (2018).
- [43] Jie Gui et al. "A Review on Generative Adversarial Networks: Algorithms, Theory, and Applications". In: *IEEE Trans. on Knowledge and Data Engineering* (2023).
- [44] M. Kang et al. "StudioGAN: A Taxonomy and Benchmark of GANs for Image Synthesis". In: *IEEE Trans. on Pattern Analysis and Machine Intelligence* (2023).
- [45] Sam Bond-Taylor et al. "Deep Generative Modelling: A Comparative Review of VAEs, GANs, Normalizing Flows, Energy-Based and Autoregressive Models". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2022).
- [46] Lei Wang et al. "A State-of-the-Art Review on Image Synthesis With Generative Adversarial Networks". In: *IEEE Access* (2020).
- [47] Jason Yosinski et al. "How transferable are features in deep neural networks?" In: *International Conference on Neural Information Processing Systems*. 2014.
- [48] Karl Weiss et al. "A survey of transfer learning". In: *Journal of Big Data* (2016).
- [49] Fuzhen Zhuang et al. "A Comprehensive Survey on Transfer Learning". In: *Proceedings of the IEEE* (2021).
- [50] Li Hu et al. "Defenses to Membership Inference Attacks: A Survey". In: *ACM Comput. Surv.* (2023).
- [51] Chris Donahue et al. "Adversarial Audio Synthesis". In: *International Conference on Learning Representations*. 2019.
- [52] Miguel Costa et al. "Shifting Capsule Networks from the Cloud to the Deep Edge". In: *ACM Trans. Intell. Syst. Technol.* (2022).

VIII. BIOGRAPHY SECTION



Miguel Costa is a Ph.D. student of electronics and computer engineering at the University of Minho, Portugal. From this same institution, Miguel received the master's degree in this academic topic, with a special focus on embedded systems and information systems and technologies. His research interests focus on artificial intelligence, artificial intelligence security on embedded systems, and reconfigurable hardware. His Ph.D. program focuses on the security of AI in microcontrollers.



Tiago Gomes received the master's degree in telecommunications engineering and the Ph.D. degree in electronics and computer engineering from the University of Minho, Braga, Portugal, in 2017. He is currently an Invited Professor with the University of Minho. His current research interests include embedded hardware acceleration for autonomous perception systems based on LiDAR sensors, and hardware/software codesign for resource-constrained Internet of Things devices.



Sandro Pinto received the Ph.D. degree in electronics and computer engineering from the University of Minho, Braga, Portugal. He is an Associate Research Professor at the University of Minho. He has a deep academic background and several years of industry collaboration focusing on operating systems, virtualization, and security for embedded, CPS, and IoT systems. He has published 70+ scientific papers in top-tier conferences/journals.